

IC TRAINING CENTER VIET NAM
FUNDAMENTAL IC DESIGN AND VERIFICATION COURSE



FINAL PROJECT
TIMER IP DESIGN SPECIFICATION

Student: Lê Quang Trung

Class: IC37

Instructor: Vũ Ngọc Tuyền

GitHub Repository:

https://github.com/quangtrungcode/TIMER_IP.git

Overleaf Project:

<https://www.overleaf.com/read/ypqrgdczgkpg#b56356>

Contents

1	Overview	3
1.1	Introduction	3
1.2	Main features	3
1.3	Block diagram	4
1.4	Interface signals	5
2	Register Specification	6
2.1	Register Summary	6
2.2	Timer Control Register (TCR)	6
2.3	Timer Data Register 0	7
2.4	Timer Data Register 1	8
2.5	Timer Compare Register 0 (TCMP0)	8
2.6	Timer Compare Register 1 (TCMP1)	9
2.7	Timer Interrupt Enable Register (TIER)	9
2.8	Timer Interrupt Status Register (TISR)	10
2.9	Timer Halt Control Status Register (THCSR)	11
3	Functional Description	12
3.1	APB slave	12
3.2	Counter Control	14
3.3	Timer Control Register	16
3.4	Counter	18
3.5	Timer Interrupt Register	20
3.6	Timer Compare Register	22
4	History	24

List of Tables

1	Interface signals of timer IP	5
2	Register Summary	6
3	Timer Control Register (TCR)	7
4	Timer Data Register 0 (TDR0)	8
5	Timer Data Register 1 (TDR1)	8
6	Timer Compare Register 0 (TCMP0)	9
7	Timer Compare Register 1 (TCMP1)	9
8	Timer Interrupt Enable Register (TIER)	10
9	Timer Interrupt Status Register (TISR)	11
10	Timer Halt Control Status Register (THCSR)	12

List of Figures

1	Block Diagram of Timer IP	4
2	APB Slave timing	13
3	APB Slave logic diagram	13
4	Counter Control timing	14
5	Counter Control logic diagram	15
6	Timer Control Register timing	16
7	Timer Control Register logic diagram	17
8	Counter timing	18
9	Counter logic diagram	19
10	Timer Interrupt Register timing	20
11	Timer Interrupt Register logic diagram	21
12	Timer Compare Register timing	22
13	Timer Compare Register logic diagram	23

1 Overview

1.1 Introduction

Timer is an essential module for every chip, used to generate accurate timing intervals or control the timing of various operations within the circuit. It can be used in various applications such as pulse generation, delay generation, event generation, PWM generation, and interrupt generation. In this project, the timer module is customized from the CLINT module of an industrial RISC-V architecture. Its primary purpose is to generate interrupts based on user settings.

1.2 Main features

The Timer IP includes the following main features:

- The IP acts as an APB slave with a 12-bit address bus.
- The register set is configured via the APB bus.
- It incorporates a 64-bit count-up counter.
- The timer uses an active low asynchronous reset.
- The counter can count based on the system clock or a divided clock (up to a divisor of 256).
- It supports a timer interrupt that can be enabled or disabled.
- The supported interrupt is a hardware, maskable, and level interrupt.
- The counter continues counting even when an interrupt or an overflow occurs.
- **Advanced Level:** Supports APB byte access, a 1-cycle wait state, and error handling for prohibited accesses.
- **Advanced Level:** The counter can be halted (stopped) in debug mode.

1.3 Block diagram

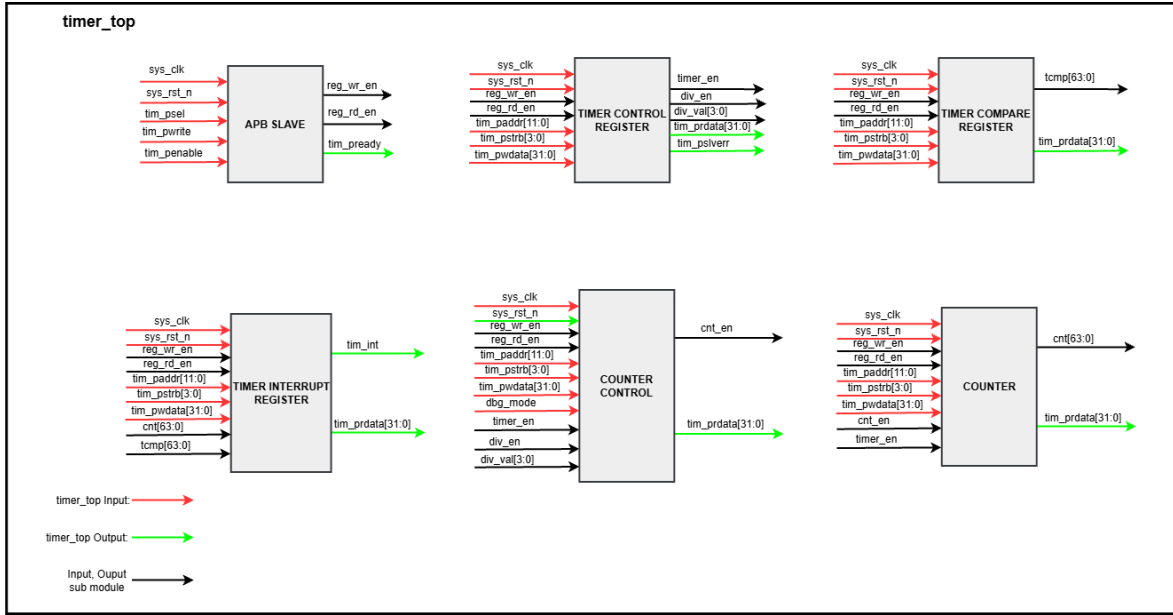


Figure 1: Block Diagram of Timer IP

The Timer IP is composed of six main sub-modules that handle the APB interface, register configurations, interrupt generation, and core counting logic. Below is the description and purpose of each sub-block:

- APB Slave:** This module acts as the communication interface between the system's APB bus and the Timer IP. It receives standard APB control signals (sys_clk, sys_rst_n, tim_psel, tim_pwrite, tim_penable) and decodes them to generate the internal read and write enable pulses (reg_rd_en, reg_wr_en). It also outputs the tim_pready signal to coordinate bus timing.
- Timer Control Register:** This block implements the Timer Control Register (TCR). It stores the user configuration that dictates how the timer operates, outputting key control signals such as timer_en to start or stop the timer, as well as div_en and div_val[3:0] to control the counting speed. It also contains the logic to issue an error response (tim_pslverr) if an invalid bus access occurs, such as attempting to modify div_val while the timer is running.
- Timer Compare Register:** This module manages the registers that store the 64-bit compare value, specifically TCMP0 and TCMP1. It outputs this 64-bit threshold (tcmp[63:0]) to the interrupt register block so the system can determine when the target time interval has been reached.
- Timer Interrupt Register:** This block encapsulates the logic for interrupt management and contains the Timer Interrupt Enable Register (TIER) and Timer Interrupt Status Register (TISR). It compares the current counter value (cnt[63:0]) against the target compare value (tcmp[63:0]) and asserts the hardware interrupt signal (tim_int) when a match occurs and the interrupt is enabled.

- **Counter Control:** This module handles the timing logic to determine exactly when the counter should increment. By taking in configuration signals like `timer_en`, `div_en`, `div_val[3:0]`, and `dbg_mode`, it calculates the correct counting frequency either the default system clock or a divided clock—and manages the halt state if the system enters debug mode. It outputs a single `cnt_en` signal to drive the core counter.
- **Counter:** This module is the core 64-bit count-up timer, corresponding to the TDR0 and TDR1 registers. Driven by the `cnt_en` signal from the Counter Control module, it increments its 64-bit value (`cnt[63:0]`) and outputs it to the bus for read accesses and to the interrupt block for comparison.

1.4 Interface signals

This section describes all the input and output ports of the Timer IP module (`timer_top`). The interface includes standard system signals, APB bus signals for configuration, and specific signals for timer operations.

Table 1: Interface signals of timer IP

Signal name	Width	Direction	Description
<code>sys_clk</code>	1	Input	System clock signal.
<code>sys_rst_n</code>	1	Input	Active low asynchronous reset signal.
<code>tim_psel</code>	1	Input	APB slave select signal.
<code>tim_pwrite</code>	1	Input	APB write control signal (High for write, Low for read).
<code>tim_penable</code>	1	Input	APB enable signal, asserted during the second cycle of an APB transfer.
<code>tim_paddr[11:0]</code>	12	Input	12-bit APB address bus.
<code>tim_pwdata[31:0]</code>	32	Input	32-bit APB write data bus.
<code>tim_prdata[31:0]</code>	32	Output	32-bit APB read data bus.
<code>tim_pstrb[3:0]</code>	4	Input	APB write strobe signal, used for byte access (Advanced level).
<code>tim_pready</code>	1	Output	APB ready signal, used to extend transfers with wait states (Advanced level).
<code>tim_pslverr</code>	1	Output	APB slave error response signal (Advanced level).
<code>tim_int</code>	1	Output	Timer hardware interrupt signal.
<code>dbg_mode</code>	1	Input	Debug mode indicator. For simplicity, testbench should not change <code>dbg_mode</code> after <code>timer_en</code> is High (during timer operation).

2 Register Specification

2.1 Register Summary

Table 2: Register Summary

Address	Abbreviation	Register name
0x00	TCR	Timer Control Register
0x04	TDR0	Timer Data Register 0
0x08	TDR1	Timer Data Register 1
0x0C	TCMP0	Timer Compare Register 0
0x10	TCMP1	Timer Compare Register 1
0x14	TIER	Timer Interrupt Enable Register
0x18	TISR	Timer Interrupt Status Register
0x1C	THCSR	Timer Halt Control Status Register
Others	Reserved	

2.2 Timer Control Register (TCR)

Offset address: 0x04

Reset value: 0x0000_0100

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				DIV_VAL[3:0]				Reserved				DIV_EN		TIM_EN	
				RW				RO				RW		RW	

Table 3: Timer Control Register (TCR)

Bit	Name	Type	Default	Description
31:12	Reserved	-	20'h0	Reserved
11:8	div_val	RW	4'b0001	Counter control mode setting: 4'b0000: Counting speed is not divided 4'b0001: Counting speed is divided by 2 (default) 4'b0010: Counting speed is divided by 4 4'b0011: Counting speed is divided by 8 4'b0100: Counting speed is divided by 16 4'b0101: Counting speed is divided by 32 4'b0110: Counting speed is divided by 64 4'b0111: Counting speed is divided by 128 4'b1000: Counting speed is divided by 256 - Others: reserved, (*) prohibit settings. When setting the prohibit value, div_val is not changed. Note: user must not change div_val while timer_en is High. (*): add hardware logic to ensure div_val is prohibited to change when timer_en is High. Access is error response in this case. (*) access is "error response" when setting prohibit value to div_val.
7:2	Reserved	RO	6'b0	Reserved
1	div_en	RW	1'b0	Counter control mode enable. 0: Disabled. Counter counts with normal speed based on system clock 1: Enabled. The counting speed of counter is controlled based on div_val Note: user must not change div_en while timer_en is High. (*): add hardware logic to ensure div_en is prohibited to change when timer_en is High. Access is error response in this case.
0	timer_en	RW	1'b0	Timer enable 0: Disabled. Counter does not count. 1: Enabled. Counter starts counting. (*) timer_en changes from H->L will initialize the TDR0/1 to their initial value.

2.3 Timer Data Register 0

Offset address: 0x04

Reset value: 0x0000_0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
TDR0[31:16]															
RW															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TDR0[15:0]															
RW															

Table 4: Timer Data Register 0 (TDR0)

Bit	Name	Type	Default value	Description
31:0	TDR0	RW	32'h0000_0000	Lower 32-bit of 64-bit counter. [Advanced level]: value of this register is cleared to initial value when timer_en changes from H->L.

2.4 Timer Data Register 1

Offset address: 0x08

Reset value: 0x0000_0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
TDR1[31:16]															
RW															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TDR1[15:0]															
RW															

Table 5: Timer Data Register 1 (TDR1)

Bit	Name	Type	Default value	Description
31:0	TDR1	RW	32'h0000_0000	Upper 32-bit of 64-bit counter. [Advanced level]: value of this register is cleared to initial value when timer_en changes from H->L.

2.5 Timer Compare Register 0 (TCMP0)

Offset address: 0x0C

Reset value: 0xFFFF_FFFF

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
TCMP0[31:16]															
RW															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TCMP0[15:0]															
RW															

Table 6: Timer Compare Register 0 (TCMP0)

Bit	Name	Type	Default value	Description
31:0	TCMP0	RW	32'hFFFFFF_FFFF	Lower 32-bit of 64-bit compare value. Interrupt pending bit is asserted when counter value is equal to compare value.

2.6 Timer Compare Register 1 (TCMP1)

Offset address: 0x10

Reset value: 0xFFFF_FFFF

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
TCMP1[31:16]															
RW															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TCMP1[15:0]															
RW															

Table 7: Timer Compare Register 1 (TCMP1)

Bit	Name	Type	Default value	Description
31:0	TCMP1	RW	32'hFFFFFF_FFFF	Upper 32-bit of 64-bit compare value. Interrupt pending bit is asserted when counter value is equal to compare value.

2.7 Timer Interrupt Enable Register (TIER)

Offset address: 0x14

Reset value: 0x0000_0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
RO															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved														INT_EN	
RO														R/W	

Table 8: Timer Interrupt Enable Register (TIER)

Bit	Name	Type	Default value	Description
31:1	Reserved	RO	31'h0	Reserved
0	int_en	R/W	1'b0	Timer interrupt enable 0: Timer interrupt is disabled. 1: Timer interrupt is enabled. When this bit is 0, no timer interrupt is output. When this bit is 1, timer interrupt pending bit can be output (interrupt pending bit is set when counter value is equals to compare value). Clearing this bit to 0 while interrupt is asserting will mask the interrupt to 0 but does not affect the interrupt pending bit (TISR.int_st bit).

2.8 Timer Interrupt Status Register (TISR)

Offset address: 0x18

Reset value: 0x0000_0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
RO															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved														INT_ST	
RO														RW1C	

Table 9: Timer Interrupt Status Register (TISR)

Bit	Name	Type	Default value	Description
31:1	Reserved	RO	31'h0	Reserved
0	int_st	RW1C	1'b0	Timer interrupt trigger condition status bit (interrupt pending bit) 0: the interrupt trigger condition does not occur. 1: the interrupt trigger condition occurred. Write 1 when this bit is 1 to clear it to 0. Write 0 when this bit is 1 has no effect. Write to this bit when it is 0 has no effect. Note1: When interrupt trigger condition occurred (counter reached compare value), counter continues to count normally. Note2: the clearance condition has highest priority.

2.9 Timer Halt Control Status Register (THCSR)

Offset address: 0x1C

Reset value: 0x0000_0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
RO															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved														HALT_ACK	HALT_REQ
RO														RO	RW

Table 10: Timer Halt Control Status Register (THCSR)

Bit	Name	Type	Default value	Description
31:2	Reserved	RO	30'h0	Reserved
1	halt_ack	RO	1'b0	[Standard level] This bit is reserved bit [Advanced level] Timer halt acknowledge 0: timer is NOT halted 1: timer is halted Timer accepts the halt request only in debug mode, indicates by debug_mode input signal
0	halt_req	RW	1'b0	[Standard level] This bit is normal R/W but has no function related. [Advanced level] Timer halt request 0: no halt req. 1: timer is requested to halt.

3 Functional Description

3.1 APB slave

The Timer IP utilizes an APB slave module to receive configuration settings and data from the system bus. This design implements the Advanced Level requirements, which include the following features:

- 12-bit width address.
- 32-bit width read/write data.
- Support for byte access.
- Support for a 1-cycle wait state to improve timing.
- Support for error handling (e.g., preventing writes to prohibited values).

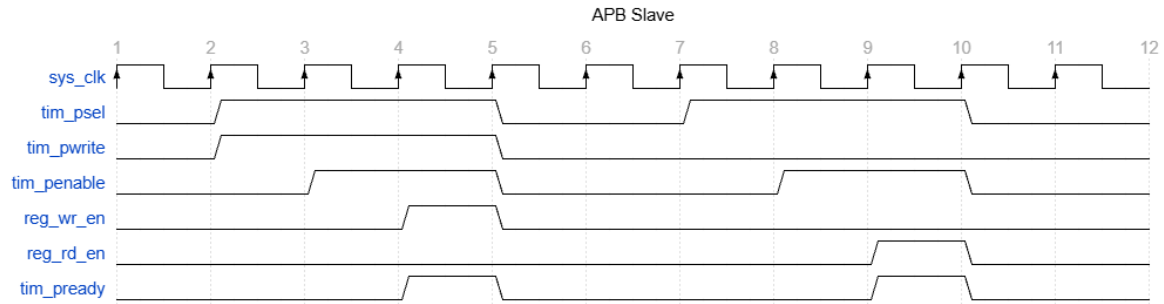


Figure 2: APB Slave timing

The waveform illustrates how the APB slave handles both write and read transactions with a 1-cycle wait state.

- **Write Transaction:** The transaction begins when `tim_psel` and `tim_pwrite` are asserted high at clock edge 2. At clock edge 3, `tim_penable` is asserted. Instead of completing the transaction immediately, the internal `reg_wr_en` and the bus `tim_pready` signals are delayed and asserted at clock edge 4, creating a single wait state before the data is successfully written into the registers.
- **Read Transaction:** A similar process occurs during a read operation starting at clock edge 7, where `tim_pwrite` remains low. After `tim_penable` goes high at edge 8, the `reg_rd_en` and `tim_pready` signals are asserted one cycle later at edge 9, holding the bus for one wait state before the read data is provided.

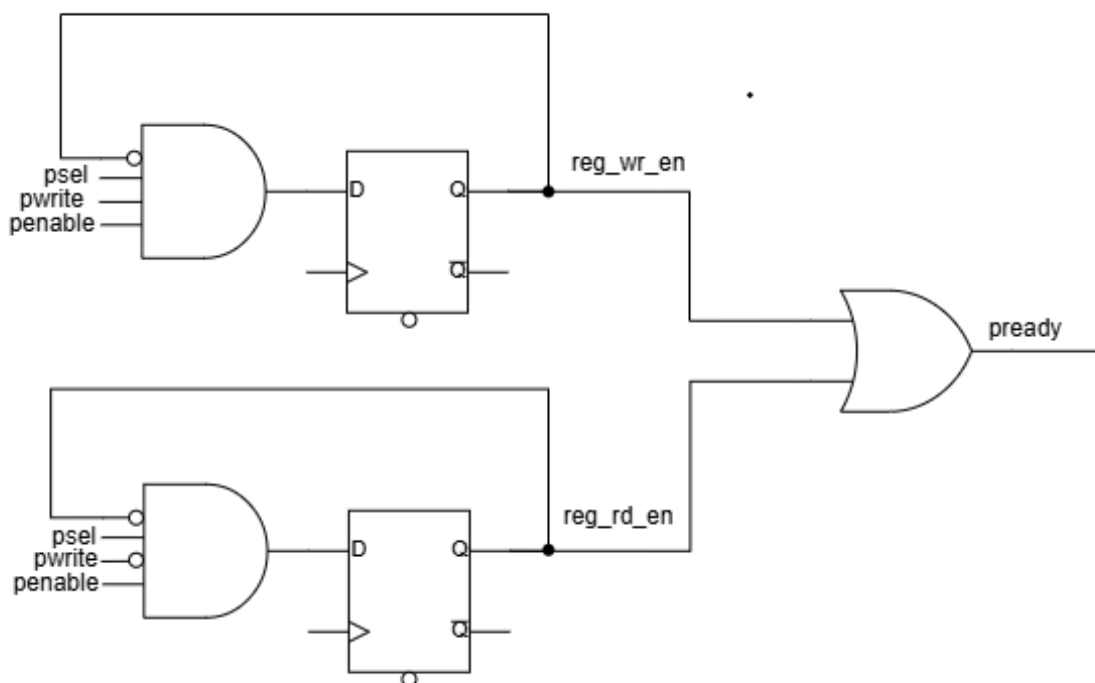


Figure 3: APB Slave logic diagram

The logic diagram demonstrates the hardware implementation used to generate the control signals and the 1-cycle wait state.

- **Write Logic:** An AND gate checks if the bus is selected (psel is high), it is a write operation (pwrite is high), the setup phase is complete (penable is high), and a write is not already in progress (inverted reg_wr_en is high). The output feeds into a D flip-flop, which delays the assertion of reg_wr_en by one clock cycle.
- **Read Logic:** Similarly, a second AND gate checks for a valid read condition (psel is high, pwrite is low, penable is high) and feeds into another D flip-flop to generate a 1-cycle delayed reg_rd_en signal.
- **Ready Signal:** The pready signal is generated by passing both reg_wr_en and reg_rd_en through an OR gate. This ensures that pready is only asserted high when either a read or write operation has completed its required 1-cycle wait state.

3.2 Counter Control

The Counter Control module is responsible for managing the precise timing of the main 64-bit counter. It calculates the correct counting frequency based on user configuration (div_val, div_en) and manages the halt execution when the system enters debug mode.

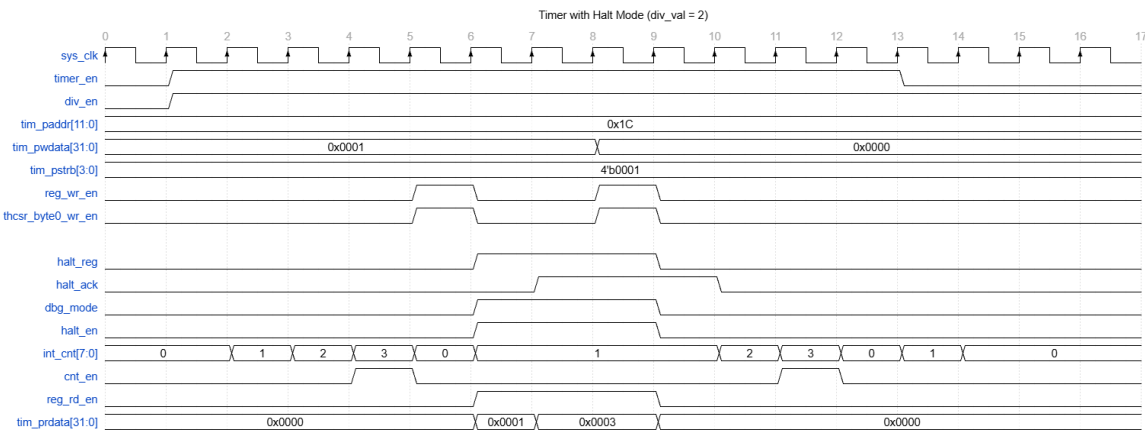


Figure 4: Counter Control timing

The waveform illustrates the operation of the Counter Control module when configured with div_val = 2 (divide-by-4 mode).

- **Normal Operation:** When both timer_en and div_en are asserted high, the internal 8-bit counter (int_cnt[7:0]) increments every clock cycle. Upon reaching the division limit (3), the module asserts the cnt_en signal for a single clock cycle to trigger the main counter.
- **Halt Request via APB Write:** At clock edge 5, an APB write transaction targets the THCSR register (address 0x1C) with data 0x0001 and strobe 4'b0001. The internal write enable (thcsr_byte0_wr_en) asserts, successfully setting halt_req high at clock edge 6.
- **Halt Execution Status Read:** Coinciding with the halt request at clock edge 6, the system enters debug mode (dbg_mode = 1), causing the internal halt_en signal to assert. This immediately freezes int_cnt at its current value (1). At

clock edge 6, an APB read operation ($\text{reg_rd_en} = 1$) at address $0x1C$ outputs $0x0001$ to $\text{tim_prdata}[31:0]$ (reflecting $\text{halt_req} = 1$ and $\text{halt_ack} = 0$). One cycle later at edge 7, the halt_ack hardware status updates to 1, dynamically changing the read data output to $0x0003$.

- **Resume Operation via APB Write:** At clock edge 8, a second write transaction to the THCSR register clears the halt request with data $0x0000$. Consequently, halt_req , dbg_mode , and halt_en fall low at clock edge 9. The internal counter seamlessly resumes incrementing from where it paused, generating the next cnt_en pulse at clock edge 11.
- **Timer Disable:** At clock edge 13, the timer_en signal is pulled low. The internal counter immediately resets its value to 0 and stops counting.

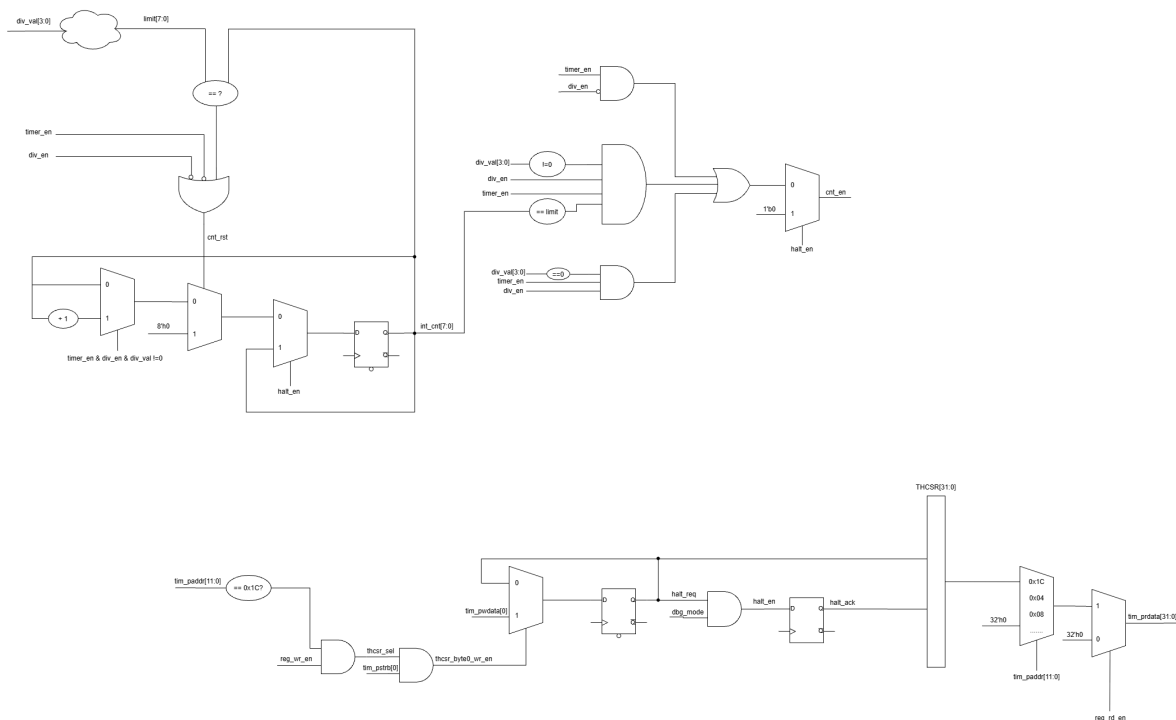


Figure 5: Counter Control logic diagram

The logic diagram details the hardware implementation used to drive the internal counter, generate the count enable pulse, and manage the halt logic.

- **Internal Counter (int_cnt) Logic:** A series of multiplexers determines the next state of the internal counter. The counter will hold its current value if halt_en is active, reset to $8'h0$ if the reset condition (cnt_rst) is met, or increment by 1 (+1) during normal operation.
- **Count Enable (cnt_en) Generation:** Combinational AND/OR logic generates the cnt_en pulse based on three distinct conditions: default mode (timer_en AND NOT div_en), division matched ($\text{int_cnt} == \text{limit}$), or when div_val equals 0. A final multiplexer ensures that cnt_en outputs $1'b0$ if halt_en is active.

- **Halt THCSR Register Logic:** The hardware decodes the address 0x1C and combines it with `reg_wr_en` and `tim_pstrb[0]` to allow writing to the `halt_req` flip-flop from the bus (`tim_pwdata[0]`). The `halt_en` signal is produced by AND-ing `halt_req` and `dbg_mode`, which then clocks a subsequent flip-flop to generate the `halt_ack` status.
- **Read Multiplexer:** A read multiplexer routes the appropriate data back to the APB bus (`tim_prdata[31:0]`) based on the address (`tim_paddr[11:0]`) whenever `reg_rd_en` is high.

3.3 Timer Control Register

The Timer Control Register (TCR) module is responsible for storing and managing the timer's core configurations, specifically `timer_en` (bit 0), `div_en` (bit 1), and `div_val` (bits 11:8). Following the Advanced Level requirements, this module supports byte-level accesses via the APB strobe signal and includes robust hardware error handling to prevent illegal configurations.

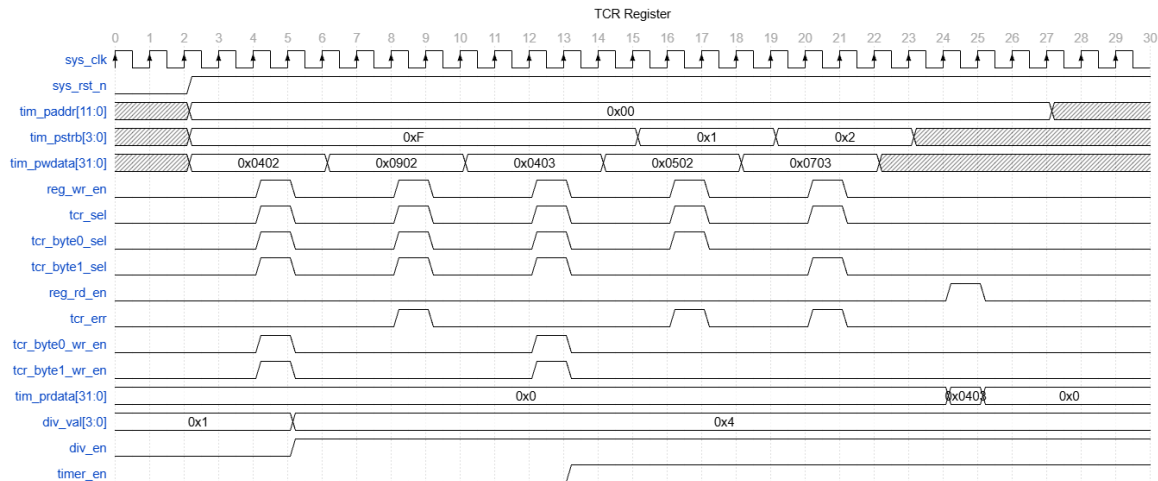


Figure 6: Timer Control Register timing

The waveform demonstrates the TCR's response to various read and write scenarios, highlighting its error-blocking capabilities:

- **Valid Write:** At clock edge 4, a write of 0x0402 with `tim_pstrb = 0xF` successfully updates `div_val` to 0x4 and `div_en` to 1.
- **Error - Prohibited Value:** At clock edge 8, the system attempts to write 0x0902. Because 0x9 is greater than the maximum allowed `div_val` of 8, the internal `tcr_err` signal is asserted, completely blocking the write. The register safely retains its previous value.
- **Enabling Timer:** At clock edge 12, a write of 0x0403 successfully sets `timer_en` to 1, which starts the timer operation.
- **Error - Modifying While Active:** At clock edges 16 and 20, the system attempts to selectively modify `div_en` and `div_val` (using individual byte strobes `tim_pstrb`).

= 0x1 and 0x2) while timer_en is already high. The tcr_err signal correctly asserts in both cases, as the configuration cannot be altered while the timer is actively counting.

- Read Operation: At clock edge 24, a read transaction targeting address 0x00 successfully outputs the current valid state of the register (0x0403) onto the tim_prdata bus.

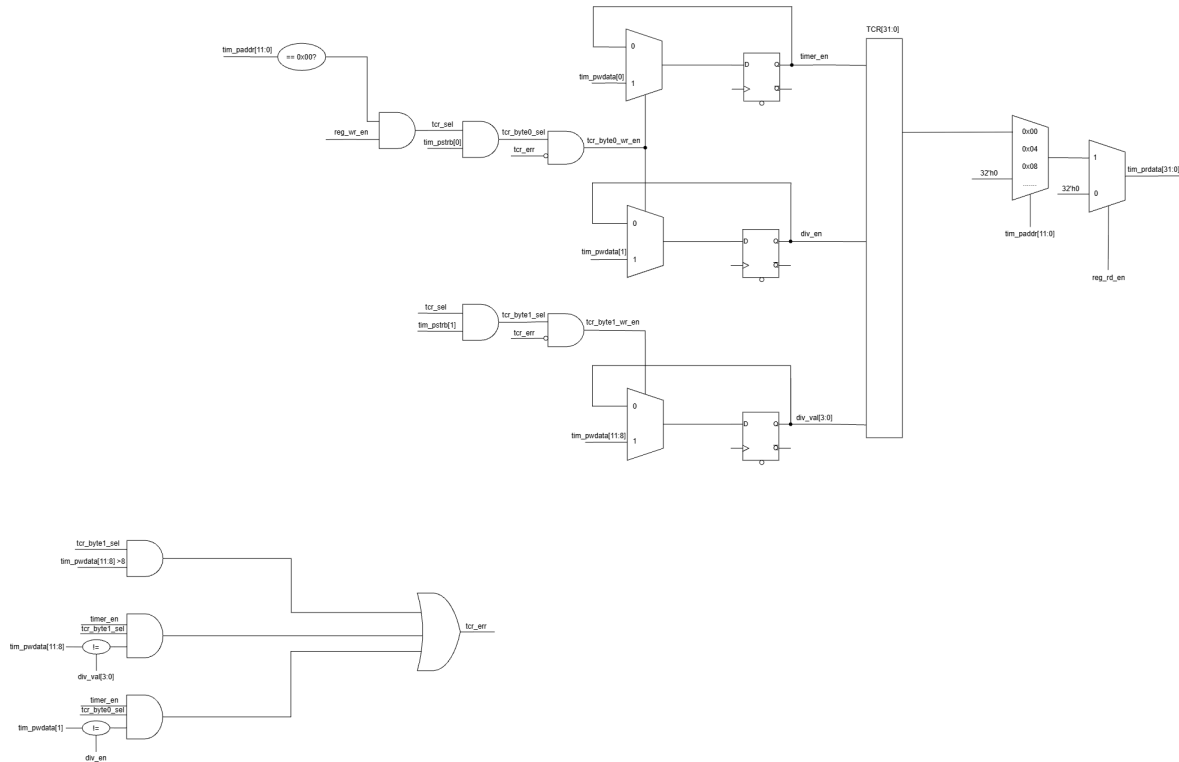


Figure 7: Timer Control Register logic diagram

The logic diagram details the hardware implementation of the TCR module, focusing on address decoding, error detection, and secure data latching:

- Address and Byte Decoding: A write operation to address 0x00 generates the tcr_sel signal. This is combined with the strobe bits (tim_pstrb[0] and tim_pstrb[1]) via AND gates to create byte-specific selection signals (tcr_byte0_sel and tcr_byte1_sel).
- Error Logic (tcr_err): A 3-input OR gate actively monitors for illegal conditions to assert tcr_err and protect the register. It triggers if:
 - The bus attempts to write a div_val greater than 8 (tim_pwdata[11:8] > 8).
 - The bus attempts to change div_val (tim_pwdata[11:8] != div_val[3:0]) while timer_en is high.
 - The bus attempts to change div_en (tim_pwdata[1] != div_en) while timer_en is high.

- **Write Enforcement:** The actual register write enables (`tcr_byte0_wr_en` and `tcr_byte1_wr_en`) are generated by AND-ing the byte selection signals with the inverted `tcr_err` signal. This masking ensures that the D flip-flops only latch new data from `tim_pwdata` if no error conditions are present.
- **Read Mux:** The outputs of the flip-flops are properly concatenated with zeros to form the full 32-bit TCR[31:0]. When `reg_rd_en` is high and the address matches `0x00`, a multiplexer routes this assembled data back to the `tim_prdata[31:0]` bus.

3.4 Counter

The Counter module contains the core 64-bit count-up logic and manages the Timer Data Registers, specifically TDR0 (lower 32 bits at address `0x04`) and TDR1 (upper 32 bits at address `0x08`). It supports normal counting operations, byte-level APB bus accesses, and an advanced hardware-level clear mechanism triggered when the timer is disabled.

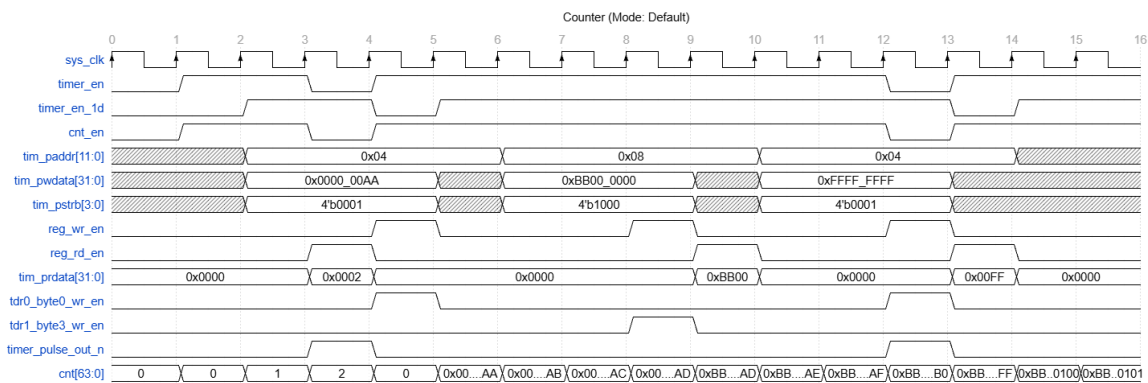


Figure 8: Counter timing

The waveform illustrates the counter's operation across different states:

- **Normal Counting:** When `timer_en` is high and `cnt_en` is asserted, the 64-bit counter (`cnt[63:0]`) increments steadily (e.g., from 0 to 1 to 2).
- **Hardware-based clear (Advanced level):** On the third clock edge, the 'timer_en' signal transitions from high to low, triggering the 'timer_pulse_out_n' signal and immediately resetting the entire 64-bit counter to its initial state of 0 on the fourth clock edge.
- **Byte-Level APB Writes:** At clock edge 4, an APB write targets TDR0 (`0x04`) with data `0x0000_00AA` and strobe `4'b0001`. This asserts `tdr0_byte0_wr_en`, updating only the lowest byte to AA. Later, at clock edge 8, a write to TDR1 (`0x08`) with strobe `4'b1000` accurately updates the highest byte (`tdr1_byte3_wr_en`), pushing the count to `0xBB...AD`.
- **Rollover Handling:** At clock edge 13, the lowest byte is forced to FF via an APB write. On the subsequent `cnt_en` pulse, the counter correctly increments `0xBB...FF` to `0xBB...0100`.

- APB Read: At clock edge 9, a read operation targeting 0x08 successfully fetches the upper 32 bits of the counter, outputting 0xBB00 onto the tim_prdata bus.

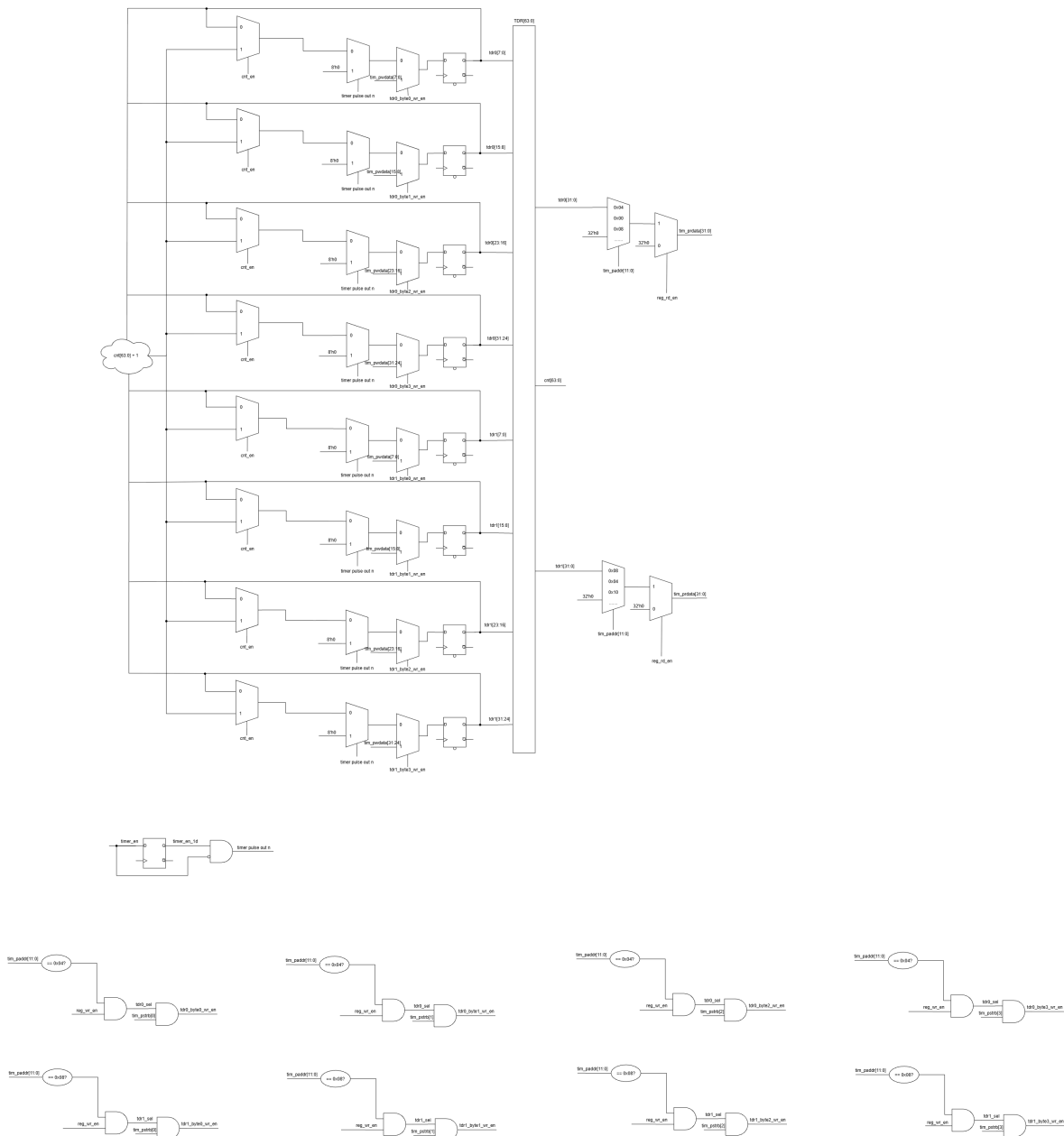


Figure 9: Counter logic diagram

The logic diagram details how the 64-bit counter is constructed using 8 parallel, byte-wide data paths to fully support byte-level APB accesses.

- Edge Detection Clear Logic: A D flip-flop delays timer_en to create timer_en_1d. Logic gates compare the inverted current timer_en state against timer_en_1d to generate timer_pulse_out_n. This creates a precise 1-cycle active-low pulse strictly on the falling edge of timer_en to clear the counter.
- Data Path Multiplexers: Each 8-bit slice of the counter is fed by a chain of three multiplexers determining its next state:

- Increment Mux: Selects between holding the current value or taking the incremented value ($\text{cnt}[63:0] + 1$) based on the `cnt_en` signal.
- Clear Mux: Takes the output of the increment mux, but forces the value to `8'h0` if `timer_pulse_out_n` goes low.
- Write Mux: Has the highest priority. It overwrites the internal state with data from the APB bus (`tim_pwdata`) if the specific byte's write enable (e.g., `tdr0_byte1_wr_en`) is asserted.
- Write Enable Decoding: Address decoding checks for `0x04` (TDR0) or `0x08` (TDR1). These selection signals are AND-ed with `reg_wr_en` and the individual bus strobe bits (`tim_pstrb[3:0]`) to isolate which specific bytes should latch new data.
- Read Multiplexers: When `reg_rd_en` is high, output multiplexers evaluate the address. If `0x04`, it routes the lower 32 bits (`cnt[31:0]`) to `tim_prdata[31:0]`; if `0x08`, it routes the upper 32 bits (`cnt[63:32]`).

3.5 Timer Interrupt Register

The Timer Interrupt Register module encapsulates the logic for generating the hardware interrupt signal (`tim_int`). It manages two key APB-accessible registers: the Timer Interrupt Enable Register (TIER) at address `0x14` and the Timer Interrupt Status Register (TISR) at address `0x18`. This module continuously monitors the counter and compare values to trigger and clear interrupts appropriately.

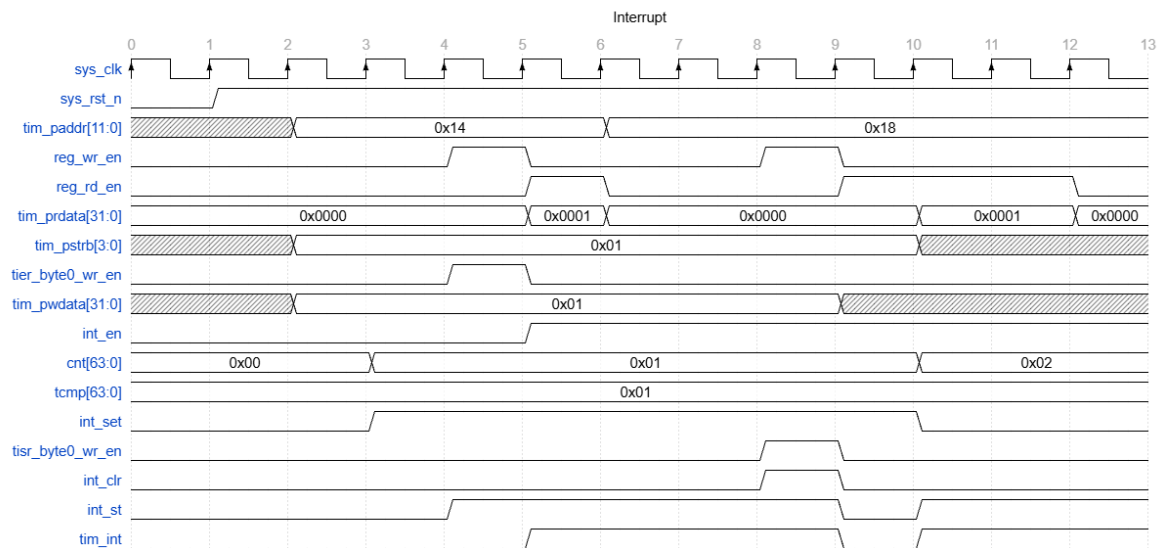


Figure 10: Timer Interrupt Register timing

The waveform illustrates the complete lifecycle of a timer interrupt, including enabling, triggering, and clearing the status:

- Enabling the Interrupt: At clock edge 4, an APB write transaction targets the TIER register (`0x14`) with data `0x01` and a valid strobe. This asserts `tier_byte0_wr_en`, safely writing a 1 to the `int_en` bit on the next clock edge.

- **Interrupt Trigger:** Between clock edges 3 and 4, the core counter (`cnt[63:0]`) increments to `0x01`, which exactly matches the predefined compare value (`tcmp[63:0] = 0x01`). This match immediately asserts the internal `int_set` signal. Consequently, the interrupt pending bit (`int_st`) goes high at clock edge 4.
- **Interrupt Output:** Because both `int_en` and `int_st` are now high, the module asserts the external hardware interrupt signal (`tim_int`) at clock edge 5.
- **Clearing the Interrupt (W1C):** At clock edge 8, the system performs a Write-1-to-Clear (W1C) operation by writing `0x01` to the TISR register (`0x18`). This asserts the internal `int_clr` signal, which forces `int_st` and `tim_int` back low at clock edge 9.

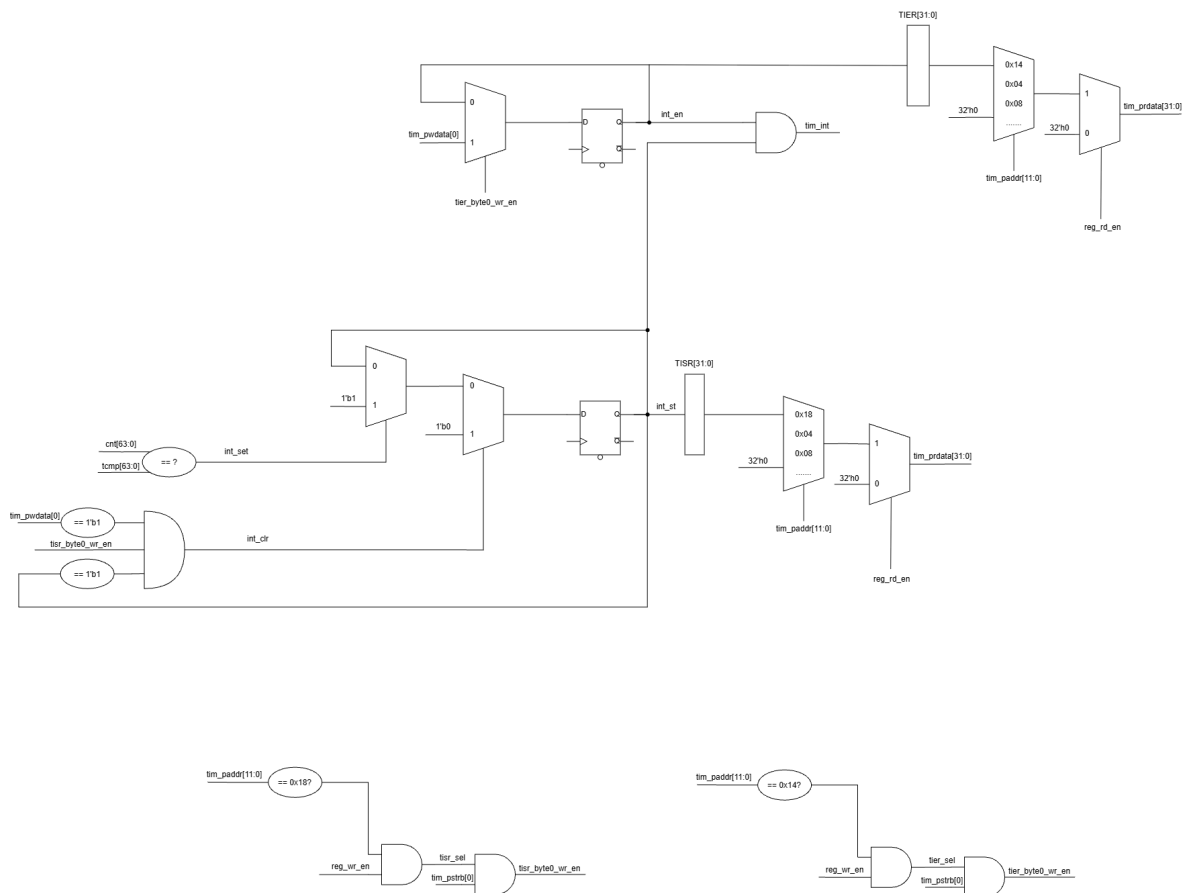


Figure 11: Timer Interrupt Register logic diagram

The logic diagram details the hardware mechanisms used to implement the interrupt rules and register accesses:

- **Interrupt Set Logic:** A 64-bit hardware comparator continuously evaluates whether `cnt[63:0]` is equal to `tcmp[63:0]`. If they match, the `int_set` signal is generated to set the pending status.
- **Interrupt Clear Logic (W1C):** A 3-input AND gate ensures that the interrupt is only cleared if three conditions are met simultaneously: a write to TISR is active (`tisr_byte0_wr_en == 1`), the specific data bit written is 1 (`tim_pwddata[0] ==`

1'b1), and the interrupt is currently pending (`int_st == 1'b1`). Writing a 0 has no effect.

- **Priority Multiplexers:** The input to the `int_st` flip-flop is controlled by two cascaded multiplexers. The first mux handles the `int_set` condition. The second mux handles the `int_clr` condition and is placed closest to the flip-flop, physically guaranteeing that the clearance condition has the highest priority over setting.
- **Output Generation:** The final `tim_int` output is strictly controlled by an AND gate combining the `int_st` (pending status) and `int_en` (enable bit). This ensures no interrupt is output if TIER disables it, even if a pending status exists.
- **Read Multiplexers:** Address decoding allows the APB bus to read the current state of `int_en` (from address 0x14) and `int_st` (from address 0x18) via the `tim_prdata[31:0]` bus when `reg_rd_en` is high.

3.6 Timer Compare Register

The Timer Compare Register module manages the 64-bit threshold value used to trigger hardware interrupts. It consists of two 32-bit registers: `TCMP0` for the lower 32 bits at address 0x0C, and `TCMP1` for the upper 32 bits at address 0x10. Both registers initialize to a default value of 0xFFFFFFFF and fully support byte-level APB accesses.

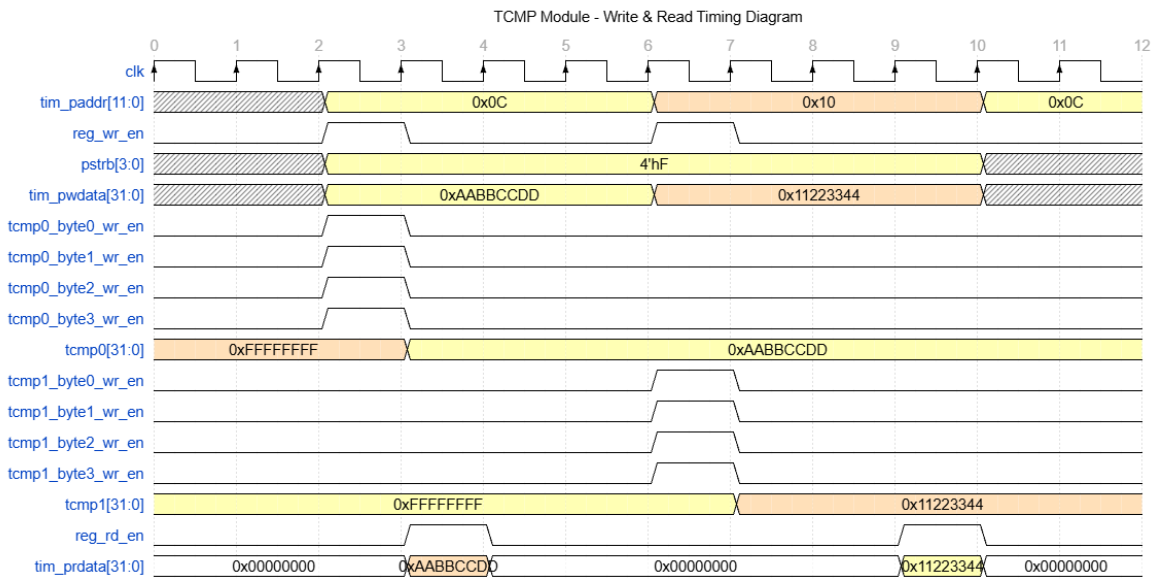


Figure 12: Timer Compare Register timing

The waveform demonstrates the write and read operations for both compare registers using full 32-bit word accesses:

- **TCMP0 Write Read:** At clock edge 2, an APB write transaction targets address 0x0C with data 0xAABBCCDD and a full byte strobe (`pstrb = 4'hF`). This simultaneously asserts all four byte-level write enables (`tcmp0_byte0_wr_en` through `tcmp0_byte3_wr_en`). At clock edge 3, `tcmp0[31:0]` successfully updates from its default 0xFFFFFFFF state to the new data. During the subsequent

read cycle at clock edge 3, `reg_rd_en` goes high, and the bus correctly outputs 0xAABBCCDD onto `tim_prdata[31:0]`.

- **TCMP1 Write Read:** At clock edge 6, a similar write transaction targets address 0x10 with data 0x11223344 and `pstrb = 4'hF`. The dedicated write enables for TCMP1 assert, updating `tcmp1[31:0]` at clock edge 7. A subsequent read operation at clock edge 9 fetches this updated value, outputting 0x11223344 to the bus.

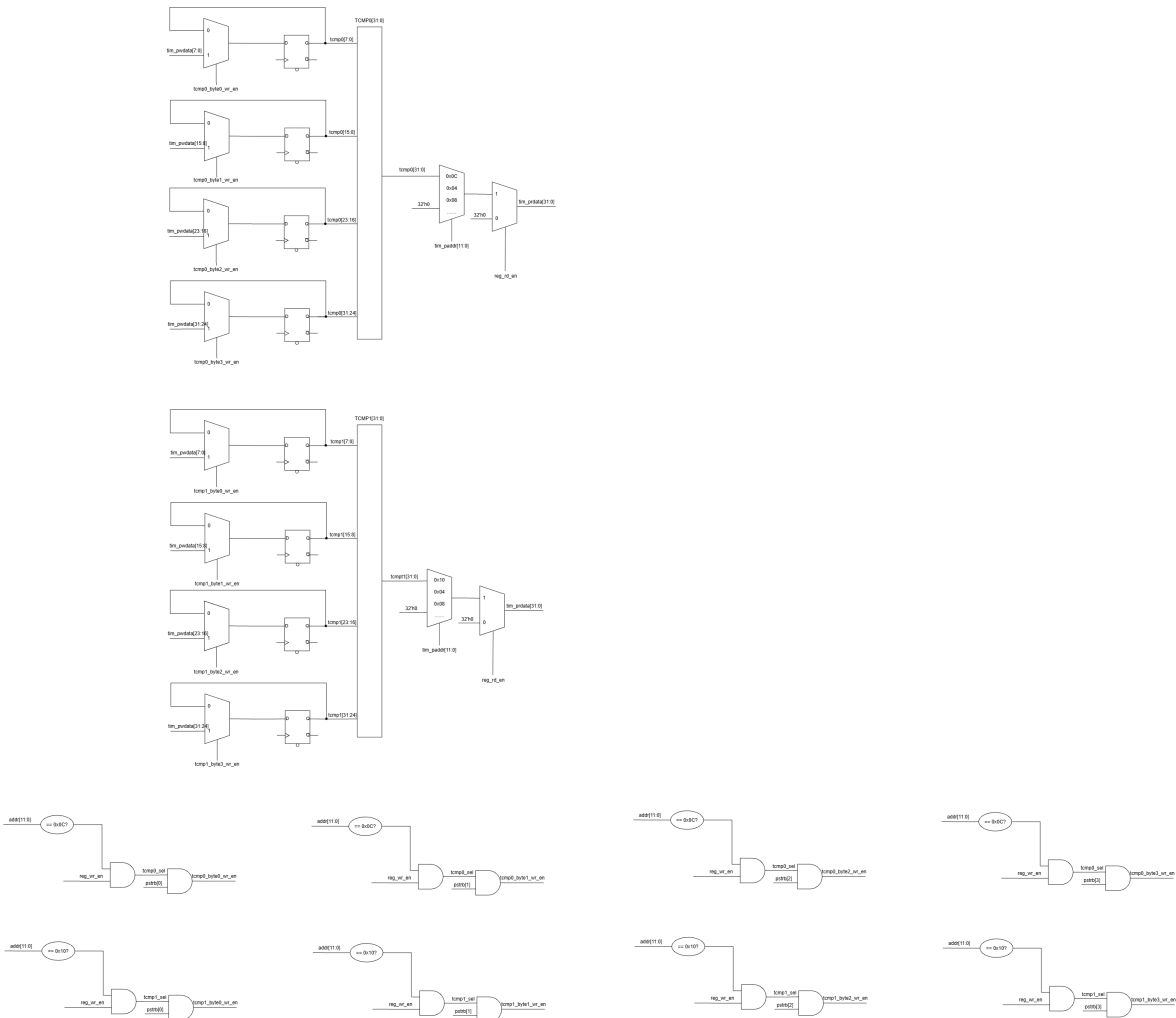


Figure 13: Timer Compare Register logic diagram

The logic diagram details the byte-sliced hardware architecture required to support advanced byte-level write operations:

- **Address and Strobe Decoding:** The hardware uses dedicated comparators to check if the bus address matches 0x0C (for TCMP0) or 0x10 (for TCMP1). This address match is AND-ed with the main `reg_wr_en` signal to create a register-specific select signal (e.g., `tcmp0_sel`). Finally, this select signal is AND-ed with individual strobe bits (`pstrb[0]` to `pstrb[3]`) to generate eight distinct byte-level write enables.
- **Data Path:** Each 32-bit register is constructed from four 8-bit multiplexer and D-flip-flop pairs. If a specific byte's write enable is high, the multiplexer routes

the corresponding 8 bits from the `tim_pwdata` bus into the flip-flop. If low, it retains its current state.

- Read Multiplexers: Two separate output multiplexers handle read requests. When `reg_rd_en` is high, they evaluate the requested address and route either `tmp0[31:0]` or `tmp1[31:0]` to the `tim_prdata[31:0]` bus, substituting zeros for unmapped addresses.

4 History

Date	Version	Description	Author
20/07/2026	1.0	Newly created	Lê Quang Trung